

UnionLabs SDR Platform — File Index & Command Mapping

Contents

File Manifest	1
Root — four files, nothing else	1
experiments/ — everything you run	1
union/ — the middleware (one, shared by every PHY and testbed)	2
drivers/ — one folder per PHY	2
Command interfaces — the CLI and the Python (sdr.py) version	2
deploy/ — Docker and install	3
docs/ — every guide, reference and PDF	3
results/ — generated output	3

File Manifest

What every file is, and where the two command interfaces map onto each other. Generated by hand against the tree as of 2026-08-17 — if a path here does not exist, this file is stale and the tree is right.

For the layout and *why* it is shaped this way, see [STRUCTURE.md](#).

Root — four files, nothing else

File	Description
README.md	Overview: the three questions every run answers, the PHYs, the doc index.
HOW_TO_ADD_ALGORITHM.md	The tutorial: write your own app.py, name your roles, link existing code.
run.sh	Run an experiment over any PHY. Also run.sh list and run.sh selftest.
radio.sh	Raw TX/RX on a USRP (B210 default; --device n210 x310).
requirements.txt	Python dependencies for everything that runs without a radio.

experiments/ — everything you run

One folder per experiment, each with an app.py (the bridge) plus its own code. This is the only folder a user needs to open.

Path
_template/app.py
_shared/fl_core.py
_shared/mnist_sgd_over_sdr.py
_shared/marl_learning.py
_shared/marl_setting.py · marl_base.py
echo/app.py
plain_echo/app.py
fl/app.py · fl/fl.py

Path

```
dl/app.py
marl/app.py + marl_env.py marl_train.py aloha_baseline.py
marl_multi/app.py + agent_node.py ap_multi.py slot_sync.py marl_multi_*.py real_channel.py marl_phy.py mock_medium.py ma
clip_semcom/app.py + semcom_core.py semcom.py phy_port.py
stc_aircomp/app.py + stc_core.py stc_aircomp.py
jammer/jammer.py
```

union/ — the middleware (one, shared by every PHY and testbed)

Path	Description
phy_link.py	The uniform contract: SdrApp, PayloadSpec, Codec, the ARQ registry, the channels (ideal/pyphy/lor), the runner
run_algo.py	Loads and adapts any experiment; resolves --algo / --channel / --role into (an app, a PHY, a node type).
driver.py	The PhyDriver interface every backend implements (transfer / broadcast / superpose), and what is uniform vs L
selftest.py	./run.sh selftest — every experiment over every radio-free PHY, one pass/fail table.

drivers/ — one folder per PHY

Path	Description
usrp/	The USRP/UHD PHY. src/ include/ (C++ engine), build/sdr_system (the modem), bi
usrp/CMakeLists.txt	Build; POST_BUILD regenerates python/sdr.py + docs/PARAMETERS.md.
lor/	The SX1276 LoRa PHY. See DEVICES.md for the physical testbed.
lor/python/lor_radio.py	One radio interface, three attachments: sim / serial / spi.
lor/python/framing.py	Fragmentation + stop-and-wait ARQ across the 255-byte MTU, with src/dst addressing
lor/python/lor_driver.py	The uniform API: LoRaChannel, LoRaLink (roles), LoRaDriver.
lor/arduino/lor_phy/lor_phy.ino	PHY firmware (Teensy 4.0 + RFM95).
lor/deploy/	inventory.sh (live Tailscale discovery), check_devices.sh, push_to_pis.sh, credentials.
lor/tools/role_selftest.py	Both link ends on threads over one simulated medium — no hardware.
sim/	The radio-free driver (implemented as a channel in union/phy_link.py).

Command interfaces — the CLI and the Python (sdr.py) version

The command files above (COMMANDS.md, COMMANDS_FEC_TESTS.txt, USRP_CARRIER_MODULATION.txt) list raw **sdr_system** CLI commands. The equivalent **Python (sdr.py)** commands — the prior interface users worked with — are shown here alongside. **Mapping rule:** every CLI --option value becomes the Python keyword option=value (hyphens -> underscores); a bare --flag becomes flag=True; the --role picks the helper (tx/rx/both/sink_arq/source_arq). Run the Python forms from drivers/usrp/python/ (from sdr import ...).

Transmit only (B210)

```
./sdr_system --role tx --tx-args serial=30CD424 --scheme QPSK --tx-gain 78 --fec true
```

```
tx(tx_args="serial=30CD424", scheme="QPSK", tx_gain=78, fec=True).run()
```

Receive only (B210)

```
./sdr_system --role rx --rx-args serial=30CD3F7 --scheme QPSK --rx-gain 20 --fec true
```

```
rx(rx_args="serial=30CD3F7", scheme="QPSK", rx_gain=20, fec=True).run()
```

Stop-and-wait ARQ pair (RX = sink, TX = source; start the sink first)

```
./sdr_system --role sink_arq --rx-args serial=30CD3F7 --scheme QPSK --fec true
./sdr_system --role source_arq --tx-args serial=30CD424 --scheme QPSK --fec true
```

```
run_pair(sink_arq(rx_args="serial=30CD3F7", scheme="QPSK", fec=True),
         source_arq(tx_args="serial=30CD424", scheme="QPSK", fec=True))
# or from a config: python3 run.py configs/qpsk_arq_pair.json
```

N210 example (exact rates + DQPSK, from USRP_CARRIER_MODULATION.txt)

```
./sdr_system --role rx --rx-args addr=192.168.20.2 --rx-subdev A:0 --rx-freq 915e6 \
--rx-rate 2e6 --tx-rate 2e6 --symbol_rate 1e6 --rx-gain 25 --scheme DQPSK

rx(rx_args="addr=192.168.20.2", rx_subdev="A:0", rx_freq=915e6,
   rx_rate=2e6, tx_rate=2e6, symbol_rate=1e6, rx_gain=25, scheme="DQPSK").run()
```

Channel sense / print-only / background

```
sense_channel(rx_args="serial=30CD3F7")          # channel_sense.py - energy sensing
tx(scheme="QPSK").command()                     # print the CLI string, don't run
source_arq(tx_args="serial=30CD424").popen()     # launch in the background
```

sdr.py exposes **every** option this way (it is auto-generated from sdr_system --help); the full option list is in PARAMETERS.md. radio.sh and run.sh wrap these for one-line use.

deploy/ — Docker and install

Path	Description
initialization.sh	Installs the toolchain; --build also compiles sdr_system + pyphy.
Dockerfile · Dockerfile.novnc · docker/	Container images and the reservation-driven launcher.
DOCKER.md	How to build and run the containers.
run_sink.sh · run_source.sh	Two-host raw radio helpers.

docs/ — every guide, reference and PDF

Path	Description
BEGINNER_GUIDE.md	Start here: install, first run, every run.sh option, adding
STRUCTURE.md	The repo map and how the two layers stack.
COMMANDS.md	Ready-to-run commands: algorithm-level, then raw T2
PARAMETERS.md	Every C++ modem option (AUTO-GENERATED — do not
SYSTEM_REFERENCE.md	The deep engine reference: the math and every algorithm
HARDWARE.md	Hardware setup and radio inventory.
MANIFEST.md	This file.
APPLICATIONS_INTRO.md · EXPERIMENT_GUIDE.md · CROSS_TESTBED_FL.md	Application intro, operating runbook, cross-testbed FL
CHANGES.md	Changelog.
.pdf	Rendered copies, built by drivers/usrp/tools/build_
sdr_stack.* · fig1_variant.* · framework_marl.* · SDR_Stack_slides.*	Architecture figures and slide decks.
make_framework_diagram.py · make_stack_variants.py	The scripts that generate those figures.
.pandoc-header.tex	LaTeX preamble used by the PDF builds (a source file,

results/ — generated output

Regenerable figures and run outputs, including phy_outputs/ (the C++ --viz target).